МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)**

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

# НАСЛЕДОВАНИЕ. ПОЛИМОРФИЗМ. АБСТРАКТНЫЕ КЛАССЫ

## Практикум

по выполнению лабораторной работы №5
по дисциплине «Программирование»

Программное обеспечение вычислительной техники и автоматизированных систем
*(указывается направленность (профиль) образовательной программы)*

09.03.04 Программная инженерия
*(указывается код и наименование направления подготовки)*

Ростов-на-Дону
2024 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко

УДК 004.432

Наследование. Полиморфизм. Абстрактные классы: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2024.

В методическом указании рассматриваются вопросы наследования в C++: назначение, принцип реализации, особенности, возможные ошибки в построения данного типа отношения между классами. Кроме этого, раскрывается механизм полиморфизма. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлению 09.03.04 «Программная инженерия», профиль «Программное обеспечение вычислительной техники и автоматизированных систем».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко,
2024
© Издательский центр ДГТУ, 2024

# 1 Теоретическая часть

## 1.1 Наследование

Наследование позволяет создавать новый класс на основе уже существующего класса. Наряду с инкапсуляцией наследование является одним из краеугольных камней объектно-ориентированного программирования.

Обычно, в связи с наследованием, которое представляет собой отношение между классами, используется следующая терминология: если класс D наследует от класса B, то говорят, что класс B — базовый для D, а D — производный от B.

Кроме повторного использования фрагментов программы, механизм наследования в объектно-ориентированных языках программирования традиционно предоставляет программисту еще одну возможность упростить процесс программирования. Эта возможность называется «полиморфизм» и позволяет производить одинаковые по смыслу действия над объектами разных классов при помощи одних и тех же выражений.

```
class Base {
private:
    int _my_precious;
protected:
    int _legacy;
public:
    void show_my_precious()
    {
        cout << _my_precious;
    };
};
class Derived: public Base {
public:
    void print_all()
    {
        cout << _my_precious; //Ошибка! Попытка доступа
                              // к приватному члену предка!
        cout << _legacy; //ОК. Защищённый член доступен из класса наследника.
        show_my_precious(); //ОК. Доступ к приватному атрибуту осуществляется
                            // через унаследованный публичный метод предка
    }
};
```

## 1.2 Виртуальные функции

Виртуальные функции класса позволяют осуществить позднее связывание. В этом случае выбор подходящей функции будет происходить уже во время выполнения программы.

Функция должна быть объявлена виртуальной в базовом классе.

```
#include <string>

class Animal {
public:
    // ...

    virtual std::string Voice() const {
        return "Generic creature voice";
    }
};

class Cat: public Animal {
public:
    // ...

    std::string Voice() const override {
        return "Meow!";
    }
};

class Dog: public Animal {
public:
    // ...

    std::string Voice() const override {
        return "Woof!";
    }
};
```

1.3 Полиморфизм

Полиморфизм позволяет обращаться с объектами разных классов так, как будто они являются объектами одного класса. Реализовать полиморфизм можно через наследование, интерфейсы и перегрузку методов. Этот подход имеет несколько весомых преимуществ:

– позволяет использовать различные реализации методов в зависимости от типа объекта, что делает код более универсальным и удобным для использования;

– уменьшает дублирование кода – можно написать одну функцию для работы с несколькими типами объектов;

– позволяет использовать общие интерфейсы и абстракции для работы с объектами разных типов;

– обеспечивает гибкость и расширяемость – можно добавлять новые типы объектов без необходимости изменять существующий код. Это дает возможность разработчикам встраивать новые функции в программу, не нарушая ее

существующую функциональность.

Полиморфизм тесно связан с абстракцией:

– абстракция позволяет скрыть детали реализации объекта и предоставить только необходимый интерфейс для работы с ним. Это помогает упростить код, сделать его более понятным и гибким;

– полиморфизм предоставляет возможность использовать один и тот же интерфейс для работы с разными объектами, которые могут иметь различную реализацию. Этот подход значительно упрощает расширение функциональности ПО.

Таким образом, абстракция позволяет определить общий интерфейс для работы с объектами, а полиморфизм позволяет использовать этот интерфейс для работы с различными объектами, которые могут иметь различную реализацию.

Рассмотрим полиморфизм на примере класса Confectionary (кондитерские изделия):

```
class Confectionary:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def describe(self):
        print(f"{self.name} по цене {self.price} руб/кг")
class Cake(Confectionary):
    def describe(self):
        print(f"{self.name} торт стоит {self.price} руб/кг")
class Candy(Confectionary):
    def describe(self):
        print(f"{self.name} конфеты стоимостью {self.price} руб/кг")
class Cookie(Confectionary):
    pass
```

В этом примере мы определяем базовый класс Confectionary, который имеет атрибуты name и price, а также метод describe(). Затем мы определяем три подкласса класса Confectionary: Cake, Candy и Cookie. Cake и Candy переопределяют метод describe() своими собственными реализациями, которые включают тип кондитерского изделия (торт и конфеты соответственно), а Cookie наследует дефолтный метод describe() от Confectionary.

## 2 Задание к лабораторной работе

Разработать программу, реализовав один из вариантов задания в соответствии с таблицей.

Базовый абстрактный класс должен содержать как абстрактные методы, так и неабстрактные, в общей сложности не менее 3 методов, если в задании указано меньшее количество методов, то недостающие придумываем самостоятельно.

Производные классы могут содержать дополнительные методы согласно заданию и/или вашей фантазии.

Коллекция объектов классов должна иметь в себе представителей от каждого класса.

При сдаче лабораторной работы могут быть заданы вопросы по теме «Наследование. Полиморфизм. Абстрактные классы».

Таблица 1. Варианты заданий

| № варианта | Задание к лабораторной работе |
|---|---|
| 1 | Создать абстрактный класс File, инкапсулирующий в себе методы Open, Close, Seek, Read, Write, GetPosition и GetLength. Создать производные классы MyDataFile1 и MyDataFile2— файлы, содержащие в себе данные некоторого определенного типа MyData1 и MyData2, а также заголовки, облегчающие доступ к этим файлам.<br><br>*Создать класс Folder, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода списка имен и длин файлов. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 2 | Создать абстрактный класс Body. На его основе реализовать классы Parallelepiped (прямоугольный параллелепипед), Cone (конус) и Ball (шар) с возможностью вычисления площади поверхности и объема.<br><br>*Создать класс Series (набор), содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 3 | Создать абстрактный класс Point (точка). На его основе создать классы ColoredPoint и Line. На основе класса Line создать класс ColoredLine и класс PolyLine (многоугольник). Все классы должны иметь виртуальные методы установки и получения значений всех координат, а также изменения цвета и получения текущего цвета. |

| | |
|---|---|
| | *Создать класс Picture, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 4 | Создать абстрактный класс Triangle (треугольник), задав в нем длину двух сторон, угол между ними, методы вычисления площади и периметра. На его основе создать классы, описывающие равносторонний, равнобедренный и прямоугольный треугольники со своими методами вычисления площади и периметра.<br><br>*Создать класс Picture, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка и получения суммарной площади. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 5 | Создать абстрактный класс Currency для работы с денежными суммами. Определить в нем методы перевода в рубли и вывода на экран. На его основе реализовать классы Dollar, Euro и Pound (фунт стерлингов) с возможностью пересчета в центы и пенсы соответственно.<br><br>*Создать класс Purse (кошелек), содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода общей суммы, переведенной в рубли, и суммы по каждой из валют. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 6 | Создать абстрактный класс Работник больницы. На его основе реализовать классы Медсестра, Хирург и Административный работник. Классы должны иметь возможность задавать и получать возраст, образование и название должности. Наряду с общими полями и методами, каждый класс должен содержать и специфичные для него поля.<br><br>*Создать класс Больница, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 7 | Создать абстрактный класс Vehicle. На его основе реализовать классы Car (автомобиль), Bicycle (велосипед) и Lorry (грузовик). Классы должны иметь возможность задавать и получать параметры средств передвижения (цена, максимальная скорость, год выпуска и т. д.). Наряду с общими полями и методами, каждый класс должен содержать и специфичные для него поля.<br><br>*Создать класс Garage, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |

| | |
|---|---|
| 8 | Создать абстрактный класс Point (точка). На его основе создать классы Квадрат и Пирамида. Все классы должны иметь методы установки и получения значений всех координат, а также получение размера.<br><br>*Создать класс Figures, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 9 | Создать абстрактный класс Figure. На его основе реализовать классы Rectangle (прямоугольник), Circle (круг) и Trapezium (трапеция) с возможностью вычисления площади, центра тяжести и периметра.<br><br>*Создать класс Picture, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 10 | Создать абстрактный класс Работник фирмы. На его основе реализовать классы Стажер, Руководящий сотрудник и Директор. Классы должны иметь возможность задавать и получать возраст, уровень допуска и название должности. Наряду с общими полями и методами, каждый класс должен содержать и специфичные для него поля.<br><br>*Создать класс Фирма, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 11 | Создать абстрактный класс Currency для работы с денежными суммами. Определить в нем методы перевода в рубли и вывода на экран. На его основе реализовать классы Dollar, Euro и Pound (фунт стерлингов) с возможностью пересчета в центы и пенсы соответственно.<br><br>*Создать класс Purse (кошелек), содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода общей суммы, переведенной в рубли, и суммы по каждой из валют. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 12 | Создать абстрактный класс Vehicle. На его основе реализовать классы Car (автомобиль), Bicycle (велосипед) и Lorry (грузовик). Классы должны иметь возможность задавать и получать параметры средств передвижения (цена, максимальная скорость, год выпуска и т. д.). Наряду с общими полями и методами, каждый класс должен содержать и специфичные для него поля. |

| | |
|---|---|
| | *Создать класс Garage, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 13 | Создать абстрактный класс Молодой человек. На его основе реализовать классы Студент, Военнослужащий и Военный курсант. Классы должны иметь возможность задавать и получать фио, возраст и сведения о военнообязанности. Наряду с общими полями и методами, каждый класс должен содержать и специфичные для него поля.<br><br>*Создать класс PeopleDB, содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 14 | Создать абстрактный класс Body. На его основе реализовать классы Parallelepiped (прямоугольный параллелепипед), Cone (конус) и Ball (шар) с возможностью вычисления площади поверхности и объема.<br><br>*Создать класс Series (набор), содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |
| 15 | Создать абстрактный класс Number с виртуальными методами, реализующими арифметические операции. На его основе реализовать классы Integer и Real.<br><br>*Создать класс Series (набор), содержащий массив/параметризованную коллекцию объектов этих классов в динамической памяти. Предусмотреть возможность вывода характеристик объектов списка. Написать демонстрационную программу, в которой будут использоваться все методы классов. |

# 3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.

– Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
– Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

– Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для Python. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
– Среды разработки и исполнения программ: PyCharm, IDLE Python, Jupyter Notebook и Anaconda.

## 4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

## 5 Контрольные вопросы к лабораторной работе

1. Что такое наследование и полиморфизм?
2. Чем наследование отличается от полиморфизма?
3. Чем отличается полиморфизм от абстракции?
4. Какие преимущества предоставляет полиморфизм в Python?
5. Как создать абстрактный класс в Python и зачем он нужен?
6. Какие основные принципы объектно-ориентированного программирования отражены в концепциях наследования, полиморфизма и абстрактных классов в Python?
7. Можно ли создать экземпляр абстрактного класса в Python? Если да, то как это сделать?

## 6 Перечень использованных информационных ресурсов

1. Широков, А. И. Информатика. Разработка программ на языке программирования Питон. Базовые языковые конструкции : учебник / А. И. Широков, М. О. Пышняк. — Москва : МИСИС, 2020. — 142 с. — ISBN 978-5-907226-76-0. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/147960 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.

2. Практикум по информатике / Н. М. Андреева, Н. Н. Василюк, Н. И. Пак, Е. К. Хеннер. — 3-е изд., стер. — Санкт-Петербург : Лань, 2024. — 248 с. — ISBN 978-5-507-47299-4. — Текст : электронный // Лань : электронно-библиотечная система. —

URL: https://e.lanbook.com/book/359810 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.

Редактор А.А. Литвинова

---

| | | |
|---|---|---|
| ЛР № 04779 от 18.05.01. | В набор | В печать |
| Объем 0,5 усл.п.л., уч.-изд.л. | Офсет. | Формат 60х84/16. |
| Бумага тип №3. | Заказ № | Тираж 75. Цена |

---

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:
344010, г. Ростов-на-Дону, пл. Гагарина, 1.